

RuntimeError Traceback (most recent call last)

Cell In[38], line 3

```
1 import seaborn as sns
----> 3 sns.regplot(x=x, y=y, logistic=True)
```

File c:\Users\My

Pc\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn\regression.py:775, in regplot(data, x, y, x_estimator, x_bins, x_ci, scatter, fit_reg, ci, n_boot, units, seed, order, logistic, lowess, robust, logx, x_partial, y_partial, truncate, dropna, x_jitter, y_jitter, label, color, marker, scatter_kws, line_kws, ax)

```
773 scatter_kws["marker"] = marker
774 line_kws = {} if line_kws is None else
copy.copy(line_kws)
--> 775 plotter.plot(ax, scatter_kws, line_kws)
776 return ax
```

File c:\Users\My

Pc\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn\regression.py:384, in

_RegressionPlotter.plot(self, ax, scatter_kws, line_kws)

```
381 self.scatterplot(ax, scatter_kws)
383 if self.fit_reg:
--> 384 self.lineplot(ax, line_kws)
386 # Label the axes
387 if hasattr(self.x, "name"):
```

File c:\Users\My

Pc\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn\regression.py:429, in

_RegressionPlotter.lineplot(self, ax, kws)

```
427 """Draw the model."""
428 # Fit the regression model
--> 429 grid, yhat, err_bands = self.fit_regression(ax)
430 edges = grid[0], grid[-1]
```

432 # Get set default aesthetics

...

192 for option in options:

193 if getattr(self, option) and not _has_statsmodels:

--> 194 raise RuntimeError(err.format(option))

RuntimeError: logistic=True requires statsmodels, an optional dependency, to be installed.

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Great — this is a **very common Seaborn error**, and your traceback already tells us the real problem 🙌

What the error means

This line:

```
python
```

```
sns.regplot(x=x, y=y, logistic=True)
```

 Copy code

uses **logistic regression** internally.

Seaborn **does not implement logistic regression itself** — it depends on another library called:

```
statsmodels
```

Since `statsmodels` is **not installed** in your Python environment, Seaborn throws this error:

```
vbnet
```

```
RuntimeError: `logistic=True` requires statsmodels, an optional dependency, to be
```

 Copy code

Fix: Install statsmodels

Open **Command Prompt / PowerShell** (not inside Jupyter) and run:

```
bash
```

```
pip install statsmodels
```

or (safer with your Python 3.14):

[Copy code](#)

```
bash
```

```
python -m pip install statsmodels
```

Wait for installation to finish.

[Copy code](#)

Then **restart Jupyter Notebook** and run your code again:

```
python
```

```
import seaborn as sns  
sns.regplot(x=x, y=y, logistic=True)
```

[Copy code](#)

If installation fails (important for Python 3.14)

Because you're using **Python 3.14**, some packages are still catching up.

If `pip install statsmodels` fails, do this:

```
bash
```

```
pip install numpy scipy pandas  
pip install statsmodels --no-build-isolation
```

Then restart Jupyter and try again.

[Copy code](#)

Why Seaborn behaves like this

Seaborn supports:

Mode	Requires
Linear regression	No extra dependency
Lowess	No extra dependency
Robust regression	statsmodels required
Logistic regression	statsmodels required

So whenever you use:

```
python
```

```
logistic=True  
robust=True
```

👉 **statsmodels is mandatory**

📋 Copy code

Quick check after installation

```
python
```

```
import statsmodels.api as sm  
print(sm.__version__)
```

If no error → you're good.

📋 Copy code

If you want, you can paste your `x` and `y` values and I can also show you the **pure scikit-learn logistic plot** version (no seaborn dependency issues).

📋 👍 💬 ↗️ ↺️ ...

logitic regresion

Great — let's learn **Logistic Regression** properly, step-by-step, in a **clear & practical way**.
(Perfect for your ML practice and interviews 👍)

What is Logistic Regression?

Despite its name, **Logistic Regression** is used for **classification**, not regression.

It predicts **probability** of a class:

0 or 1

Yes or No

Pass or Fail

Spam or Not Spam

The Core Idea

Linear regression gives any real value:

ini

$$y = wx + b$$

Logistic regression wraps this with the **Sigmoid function**:

 Copy code

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

So final model becomes:

$$\hat{y} = \frac{1}{1 + e^{-(wx+b)}}$$

This converts output into a **probability between 0 and 1**.

Example Problem

Will the student pass?

Hours Studied	Result
1	0
2	0
3	0
4	1
5	1
6	1

Logistic Regression in Python (scikit-learn)

python

```
import numpy as np
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt

# Data
X = np.array([1,2,3,4,5,6]).reshape(-1,1)
y = np.array([0,0,0,1,1,1])

# Train model
model = LogisticRegression()
model.fit(X, y)

# Prediction
x_test = np.linspace(1, 6, 100).reshape(-1,1)
y_prob = model.predict_proba(x_test)[:,-1]

# Plot
plt.scatter(X, y)
plt.plot(x_test, y_prob)
plt.xlabel("Hours studied")
```

```
plt.ylabel("Probability of Passing")  
plt.show()
```

You'll see the famous **S-shaped curve**.



Decision Boundary

By default:

vbnet

Probability $\geq 0.5 \rightarrow$ Class 1

Probability $< 0.5 \rightarrow$ Class 0

 Copy code



Loss Function (Very Important for Interviews)

Logistic Regression uses:

Log Loss / Binary Cross-Entropy

$$Loss = -[y \log(p) + (1 - y) \log(1 - p)]$$



Why not Mean Squared Error?

Because MSE creates **non-convex loss** for classification $\rightarrow$ slow & unstable learning.

Log Loss is **convex** $\rightarrow$ stable & fast convergence.



Advantages

- ✓ Works well for linearly separable data
 - ✓ Outputs probabilities
 - ✓ Very interpretable
 - ✓ Fast training
-

Limitations

- ✗ Only linear decision boundary
 - ✗ Sensitive to outliers
 - ✗ Not good for complex patterns
-

Interview One-Line Definition

Logistic Regression is a supervised learning algorithm used for binary classification that models the probability of an event using the sigmoid function.

If you'd like, next I can explain:

- ✓ Multiclass Logistic Regression
- ✓ How gradient descent works in logistic regression
- ✓ Or implement it **from scratch** in Python (no libraries)

Which one do you want?



explain formula wit all dependencies

Excellent — this is the **real core of Logistic Regression**.

I'll explain **every formula + every dependency** clearly and completely.

Read this slowly — after this, logistic regression will feel *obvious*.

1 Model Equation (Foundation)

We start exactly like linear regression:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

or in vector form:

$$z = \mathbf{w}^T \mathbf{x} + b$$

Dependencies here:

Symbol	Meaning
x_i	input features
w_i	model weights
b	bias
z	linear combination (real number, $-\infty$ to $+\infty$)



2

Sigmoid Function (Probability Generator)

We convert z into probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

So final prediction becomes:

$$\hat{y} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

Dependencies:

Term	Meaning
e	Euler's number (≈ 2.718)
$\hat{y}$	predicted probability
Range	$0 < \hat{y} < 1$



3

Decision Rule

If $\hat{y} \geq 0.5 \Rightarrow$ Class 1

Else Class 0



Loss Function (Log Loss / Binary Cross Entropy)

For one data point:

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

For entire dataset of m samples:

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Dependencies:

Symbol	Meaning
$y^{(i)}$	true label
$\hat{y}^{(i)}$	predicted probability
m	number of samples
$\log$	natural logarithm



Optimization: Gradient Descent

We minimize the cost by updating parameters.

Partial derivatives:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})$$

Update rules:

$$w_j := w_j - \alpha \frac{\partial J}{\partial w_j}$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

Dependencies:

Symbol	Meaning
α	learning rate
m	number of samples
$\hat{y} - y$	prediction error



Why the magic works

Because:

$$\frac{d}{dz} \sigma(z) = \sigma(z)(1 - \sigma(z))$$

This clean derivative makes the entire optimization **stable and efficient**.



Final Pipeline (Complete Dependency Chain)

lua

$x \rightarrow z = wx + b \rightarrow \text{sigmoid}(z) \rightarrow \hat{y} \rightarrow \text{loss} \rightarrow \text{gradients} \rightarrow \text{updated } w, b \rightarrow \text{repeat}$

 Copy code

If you want next, I can show:

- ◆ Full derivation of gradients step-by-step
- ◆ How decision boundary is formed geometrically
- ◆ Logistic Regression from scratch code (no sklearn)

Just tell me which one 😊



sigmod function

Perfect — the **Sigmoid function** is the heart of Logistic Regression.

Let's break it down **slowly, mathematically, and intuitively** 🙌



Sigmoid Function



Formula

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Every dependency explained

Symbol	Meaning
z	Any real number (output of linear model $w x + b$)
e	Euler's constant ≈ 2.71828
$\sigma(z)$	Output between 0 and 1 (probability)



Why Sigmoid is needed

Linear model gives:

$$z = w x + b \Rightarrow (-\infty, +\infty)$$

Classification needs:

$$0 \leq \text{probability} \leq 1$$

Sigmoid **compresses** any real number into **(0,1)**.



How Sigmoid behaves

z	$\sigma(z)$
$-\infty$	0
-3	0.047
-1	0.269
0	0.5
1	0.731
3	0.953
$+\infty$	1

So:

- large negative $\rightarrow$ class 0
- large positive $\rightarrow$ class 1
- zero $\rightarrow$ uncertain (50%)

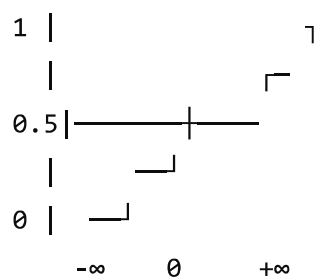


Geometric intuition

Sigmoid creates the famous **S-shaped curve**:

markdown

Probability



Critical derivative

[Copy code](#)

Used in training:

$$\frac{d}{dz}\sigma(z) = \sigma(z)(1 - \sigma(z))$$

This makes gradient descent **mathematically stable and fast**.

Why not any other function?

Sigmoid:

- ✓ smooth
 - ✓ differentiable everywhere
 - ✓ maps to probability
 - ✓ simple derivative
-

Tiny example

If:

$$z = 2$$
$$\sigma(2) = \frac{1}{1 + e^{-2}} = \frac{1}{1 + 0.135} = 0.881$$

→ 88.1% chance of class 1

If you'd like, next I can explain:

- ◆ Why sigmoid causes vanishing gradients
- ◆ Comparison: Sigmoid vs Softmax vs Tanh
- ◆ How sigmoid shapes the decision boundary

Just say the word 😊



